

Task 18 – Intro to PyTorch

Tensors, Autograd & Building a Simple Neural Network

PKCERT AI & Software Development Internship

Prepared by: Abdul Rehman | Dataset: Breast Cancer Wisconsin | PyTorch 2.13.0

1. Objective

This task builds practical fluency with PyTorch's tensors, its automatic differentiation engine (autograd), and the `nn.Module` API, mapping each concept back to the manual NumPy derivations from Task 16 to reinforce the underlying mathematics.

2. Tensors & Basic Operations

Tensors were created using four methods — `torch.tensor`, `torch.zeros/ones`, `torch.arange`, and conversion from a NumPy array via `torch.from_numpy` — each suited to a different situation: fixed data, placeholder initialization, sequential values, and bridging existing NumPy data respectively.

`view()` requires contiguous memory and shares storage with the original tensor; `reshape()` is more flexible and copies data when necessary. Broadcasting was demonstrated by adding a shape-(4,) vector across a (3,4) matrix, where the smaller shape is stretched across the larger one automatically.

Operation	Time (1000×1000 matmul)
NumPy	0.0257 sec
PyTorch (CPU)	0.0301 sec

On CPU, both libraries call similar underlying BLAS routines, so timing is comparable for a single matrix multiply; PyTorch's advantage typically appears on GPU or in chained/batched operations.

`torch.from_numpy()` and `.numpy()` share the same underlying memory buffer rather than copying — modifying a tensor created this way was confirmed to modify the source NumPy array in place, and vice versa.

3. Autograd

PyTorch builds its computational graph dynamically as operations execute ("define-by-run") — each operation on a tensor with `requires_grad=True` is recorded, and `.backward()` walks the graph in reverse to compute gradients.

Check	Result
$f(x) = 3x^2 + 2x + 1$ at $x = 4$	$f(x) = 57.0$
Autograd gradient df/dx	26.0
Hand-derived gradient $(6x + 2)$	26.0 — match confirmed

Gradients accumulate by default across multiple `.backward()` calls rather than being overwritten, which is why `zero_grad()` must be called before each new training step — otherwise gradients from prior steps leak into the current update.

`torch.no_grad()` is used to disable tracking for a whole block (e.g. manual weight updates, evaluation/inference); `.detach()` is used to strip history from one specific tensor while keeping its value (e.g. for logging).

Autograd vs. manual backpropagation cross-check

Method	dL/dw
Manual (NumPy, Task 16 style)	-0.158631278
Autograd (PyTorch)	-0.158631265

Maximum numerical difference: 1.32×10^{-8} — well within the 1×10^{-6} tolerance, confirming autograd reproduces the manually derived gradient.

4. Building & Training the Neural Network

A feed-forward network was built as an `nn.Module` subclass with one hidden layer (input → 16 units, ReLU → output, 2 classes), trained on the Breast Cancer Wisconsin dataset (same dataset family as Task 16/15) with an 80/20 train-test split and standardized features.

Configuration

Component	Choice	Justification
Loss function	CrossEntropyLoss	Standard for classification with raw logit outputs
Optimizer	Adam (lr = 0.01)	Faster, more stable convergence than plain SGD
Hidden layer size	16 units	Sufficient capacity for a 30-feature tabular dataset
Epochs	100	Loss plateaued well before this point

The training loop followed the standard PyTorch pattern: forward pass → loss computation → `loss.backward()` → `optimizer.step()` → `optimizer.zero_grad()`, with the loss tracked and plotted over all 100 epochs (see notebook for the loss curve).

Test set evaluation

Metric	Value
Accuracy	0.9561
Precision	0.9855
Recall	0.9444
F1-score	0.9645

A confusion matrix heatmap was generated to visualize class-level performance (see notebook).

Model comparison

Implementation	Accuracy	Training time
Manual NumPy (Task 16)	— fill in from Task 16 —	— fill in from Task 16 —
PyTorch (this task)	0.9561	< 1 sec (100 epochs, CPU)
scikit-learn MLPClassifier	0.9649	0.4445 sec

Hyperparameter experiment — learning rate

Learning rate	Final loss (epoch 100)	Test accuracy
0.1	0.0003	0.9561
0.01	0.0178	0.9649
0.001	0.1377	0.9561

lr = 0.1 drove training loss lowest but did not improve test accuracy over lr = 0.01, suggesting mild overfitting at the highest rate; lr = 0.001 converged too slowly within 100 epochs. lr = 0.01 gave the best balance of convergence speed and generalization.

5. Analysis & Documentation

Trained parameters were saved with `torch.save(model.state_dict())` and successfully reloaded via `load_state_dict()`, with the reloaded model reproducing the original 0.9561 test accuracy exactly — confirming correct persistence.

Manual NumPy (Task 16) vs. PyTorch — key differences

- Code complexity: Task 16 required manually coding the forward pass, loss derivative, and backpropagation through every layer; PyTorch reduces this to defining `forward()` once, with autograd handling differentiation automatically.
- Ease of experimentation: swapping the optimizer, learning rate, or architecture in PyTorch is a one-line change, whereas the manual implementation would require rewriting the update rule and gradient calculations by hand.

What autograd abstracts away

Autograd abstracts away the chain-rule bookkeeping — tracking which operations ran, in what order, and how to differentiate each to get gradients for every parameter. The manual exercise in Task 16 remains valuable because it builds the underlying intuition for why the network learns, rather than treating `.backward()` as a black box.

A limitation encountered

A common gotcha was a shape/type mismatch with `CrossEntropyLoss`, which expects raw logits of shape `(batch, num_classes)` and integer class labels of shape `(batch,)` — not one-hot vectors. This was resolved by keeping labels as `torch.long` class indices and passing unmodified (non-softmaxed) logits directly into the loss function.

Full code, outputs, and plots (loss curve, confusion matrix, learning-rate comparison) are available in the accompanying Jupyter notebook: `Task18_Abdul_Rehman`